RAG Study Buddy

A Retrieval-Augmented Generation chatbot that answers questions from your own study documents, with source citations

Capstone Project Report

From Tokens to Transformers: A Course on GenAI x LLM Mastery

Consulting & Analytics Club, IIT Guwahati

Submitted by: Shital Asawal

Date: 19 October 2026

GitHub: https://github.com/Shital1414/rag-study-buddy

1. Abstract

Large Language Models (LLMs) write fluent text but can state wrong facts with confidence, and they know nothing about a student's own notes. This project builds **RAG Study Buddy**, a Retrieval-Augmented Generation system that answers questions using only the documents the user uploads (PDF, TXT or Markdown). Documents are split into overlapping chunks, indexed with TF-IDF (unigrams and bigrams) and searched with cosine similarity. The best passages are given to an LLM (Gemini or Groq, optional) or to an offline extractive answerer, and every answer lists the passages it came from. If no passage is similar enough, the system refuses instead of guessing.

On a 24-question test set built from the sample notes, the default configuration (110-word chunks) achieved **Hit@1 = 0.79**, **Hit@3 = 1.00** and **MRR = 0.89**, and it correctly refused **6 of 6** off-topic questions. The complete system runs with no API key and no GPU, and comes with a Streamlit web interface and a reproducible evaluation script.

2. Problem Statement and Motivation

Students collect lecture notes, slides and papers, but finding a specific answer inside them is slow. A general chatbot does not know these private documents and may hallucinate. The goal is a study assistant that is:

- **Grounded** - answers come from the user's documents, not from the model's memory.
- **Verifiable** - every answer shows the source passages and similarity scores.
- **Honest** - it says it cannot find the answer when the documents do not cover the question.
- **Accessible** - it runs on a laptop for free, with an optional free LLM key for better wording.

3. Background: Concepts from the Course Used Here

Concept	How it appears in this project
Tokenization	TF-IDF tokenizes text into words and word pairs (bigrams); chunk size is measured in words. LLMs use sub-word tokens.
Vector representations	Each chunk and question becomes a sparse TF-IDF vector. Similarity is the cosine of the angle between vectors, the same measure used for dense embeddings.
Retrieval-Augmented Generation	Retrieve the most relevant chunks first, then generate an answer grounded in them (Lewis et al., 2020).
Prompt engineering	The prompt restricts the LLM to the given context, demands a fixed refusal sentence, limits length and asks for [1], [2] citations.
Hallucination control	Grounding, low temperature (0.2), a similarity threshold that triggers refusal, and visible sources.
Evaluation	Hit@k, Mean Reciprocal Rank and out-of-scope refusal rate, plus a chunk-size ablation.

4. System Design

The system has an indexing phase and a query phase, as in a standard RAG pipeline.

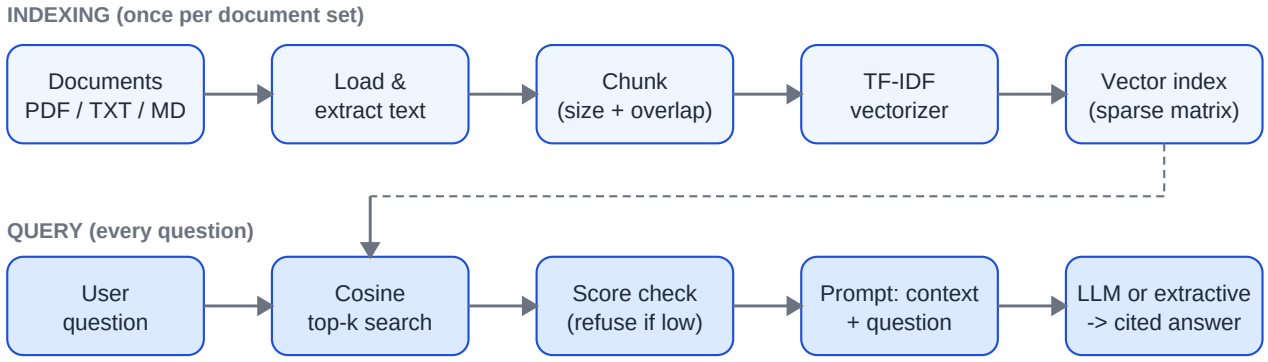

Figure 1. RAG Study Buddy architecture (dashed line: the index is used by the search step).

4.1 Modules

File	Responsibility
rag.py	Loading (txt, md, pdf), chunking, TF-IDF retrieval, prompt building, Gemini/Groq calls, extractive fallback, refusal logic.
app.py	Streamlit interface: file upload, settings sliders, chat history, expandable sources, optional API key.
evaluate.py	Retrieval metrics and chunk-size ablation; writes results.json.
make_report.py	Builds this PDF from the live system and the saved results.
data/llm_notes.txt	Sample study notes (about 1,500 words) on NLP, Transformers, LLMs, RAG, agents and evaluation.

5. Implementation Details

5.1 Chunking

Text is split on blank lines into paragraphs. Paragraphs are packed into a chunk until the word limit is reached, so a chunk usually keeps one idea together. A paragraph longer than the limit is cut with a sliding window that overlaps the previous piece by a set number of words, so a sentence at a boundary still appears whole in one chunk. Defaults: 110 words, 25 overlap.

5.2 Retrieval

A scikit-learn `TfidfVectorizer` (English stop words removed, unigrams plus bigrams, sub-linear term frequency) indexes all chunks. A question is transformed with the same vectorizer and compared to every chunk by cosine similarity; the top-k chunks (default 3) are returned with their scores. TF-IDF was chosen because it needs no model download, is fast, and is easy to explain.

5.3 Refusal to answer (hallucination control)

If the best similarity score is below 0.08, the system returns a fixed message that the answer is not in the documents. This threshold was chosen by inspecting scores of in-scope versus off-topic questions on the sample notes; it is a tunable constant (`MIN_SCORE`).

5.4 Answer generation

- **LLM mode** (Gemini or Groq key provided): the prompt contains the numbered passages and question, tells the model to use only that context, to reply with the refusal sentence if the answer is missing, and to cite [1], [2]. Temperature is 0.2. If the API call fails, the app falls back to extractive mode and shows a warning.
- **Extractive mode** (no key): the top chunks are split into sentences, sentences are ranked by TF-IDF similarity to the question, and the best ones (at most three, dropping those far less relevant than the best) are returned in reading order with their source numbers.

5.5 Web interface

The Streamlit app lets the user upload several files, adjust top-k, chunk size and overlap, choose a provider, and chat. Each answer has a Sources expander showing the passages and similarity scores. The index is cached so it is rebuilt only when documents or chunk settings change.

6. Evaluation

6.1 Setup

The test set has **24 in-scope questions**, each paired with a phrase that must appear in a correct passage, and **6 out-of-scope questions** (for example, cricket and recipes) that the system should refuse. A retrieved chunk counts as correct if it contains the evidence phrase. Metrics:

- **Hit@1 / Hit@3** - fraction of questions where a correct chunk is ranked first / within the top 3.
- **MRR** - mean of 1/rank of the first correct chunk (higher is better, maximum 1).
- **Refusal rate** - fraction of off-topic questions for which the system answers "not found".

6.2 Results and chunk-size ablation

Chunk size	Overlap	# chunks	Hit@1	Hit@3	MRR	Refusal
50 words	10	52	0.50	0.92	0.69	1.00
110 words (default)	25	17	0.79	1.00	0.89	1.00
200 words	40	8	0.83	1.00	0.92	1.00
350 words	60	5	0.88	1.00	0.94	1.00

Table 1. Retrieval results on the sample notes (k = 3).

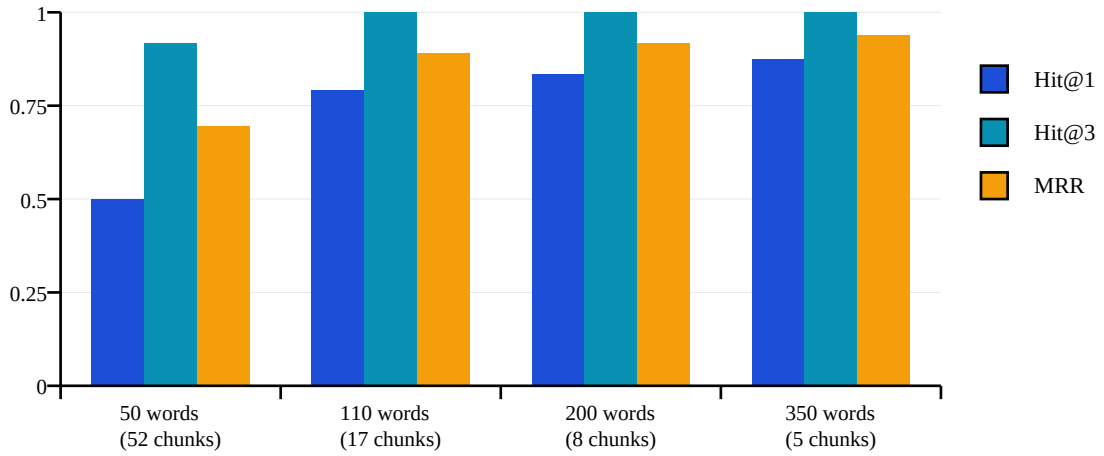

Figure 2. Retrieval metrics by chunk size.

6.3 Analysis

- **Very small chunks (50 words) hurt.** Hit@1 falls to 0.50 because an answer is often split across chunks; two questions ("Why do LLMs hallucinate?" and "Which vector databases are mentioned?") were missed even at rank 3.
- **110 words is a good balance.** It finds the right passage for all 24 questions within the top 3 and ranks it first for 19 of them, while keeping passages short enough to read.
- **Bigger chunks score higher here, but be careful.** With only 5 chunks in total, a top-3 search covers 60% of the whole document, so Hit@3 = 1.00 is almost guaranteed and says little. Large chunks also carry more irrelevant text into the LLM prompt. For this reason 110 words was kept as

the default.

- **All off-topic questions were refused at every chunk size**, showing the similarity threshold works on this document.

6.4 Limitations of this evaluation

The test questions were written by the project author using the same notes, so scores are likely higher than for real users. The notes are short (about 1,500 words), and the set is small, so differences of one question move Hit@1 by about 4 percentage points. Answer quality (faithfulness and relevance) was checked manually on a few examples only, and the LLM mode (Gemini/Groq) was not part of the automated evaluation because it needs an external API key.

7. Sample Interactions

Real output of the offline (extractive) mode on the sample notes, using the default settings:

Q: What is Retrieval-Augmented Generation?

A: Hallucinations can be reduced by grounding the model in external documents, using Retrieval-Augmented Generation, lowering the temperature and asking the model to cite sources. [1] Retrieval-Augmented Generation (RAG) combines a retrieval system with an LLM. [2]

Mode: extractive | top similarity 0.37 | sources: [1] chunk 9, [2] chunk 10, [3] chunk 15

Q: How does self-attention work?

A: A Transformer is built from stacked layers, each containing a multi-head self-attention block and a feed-forward network, with residual connections and layer normalization. [1] Self-attention lets every token in a sequence look at every other token and decide how much to focus on each. [2]

Mode: extractive | top similarity 0.25 | sources: [1] chunk 3, [2] chunk 4, [3] chunk 14

Q: What is the difference between fine-tuning and RAG?

A: Fine-tuning changes the model weights by training on extra data, and is best for teaching a style, format or specialised behaviour. [1] Fine-tuning is more expensive and harder to update, while RAG is cheaper and easier to keep current. [1] In supervised fine-tuning (SFT), the model is trained on curated examples of instructions and good responses. [3]

Mode: extractive | top similarity 0.30 | sources: [1] chunk 16, [2] chunk 15, [3] chunk 6

Q: Who won the 2011 cricket world cup?

A: I couldn't find that in the uploaded documents. Try rephrasing, or upload a document that covers this topic.

Refused: top similarity below the threshold, so no answer was invented.

Note: in extractive mode, the last cited sentence is sometimes only loosely related to the question. An LLM answers more fluently and filters such sentences, which is why LLM mode is offered as an option.

8. Limitations and Future Work

- **Keyword matching, not meaning.** TF-IDF cannot match a question that uses different words from the document (for example "car" versus "automobile"). Next step: dense embeddings (sentence-transformers) stored in FAISS or Chroma, and hybrid search combining both.
- **No OCR.** Scanned PDFs without a text layer cannot be read.
- **Single-step retrieval.** Complex questions that need several documents could use a multi-agent design: a planner agent that splits the question, a retriever agent, and a writer agent that checks citations (the Multi-Agent Systems topic from the course).
- **Evaluation.** A larger and independently written question set, plus faithfulness scoring with an LLM judge, would give more reliable numbers.
- **Study features.** Quiz and flashcard generation from the retrieved passages, and conversation memory for follow-up questions.

9. Conclusion

RAG Study Buddy shows the full RAG workflow end to end: load, chunk, index, retrieve, generate and cite. Its main design choices (grounded prompts, visible sources and a refusal threshold) target the main weakness of plain LLMs, which is confident but unsupported answers. The chunk-size experiment showed that chunking has a measurable effect on retrieval quality, and the project runs fully offline so anyone can reproduce the results. The main lesson is that the quality of a RAG system depends as much on retrieval and evaluation as on the language model.

10. How to Run

```
pip install -r requirements.txt
streamlit run app.py      # web app at http://localhost:8501
python evaluate.py        # reproduce Table 1
```

Optional: choose Gemini or Groq in the sidebar and paste a free API key, or set GEMINI_API_KEY / GROQ_API_KEY.

References

- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
- Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.
- Salton, G. and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management.
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12.
- Streamlit documentation, https://docs.streamlit.io